A Methodology for Generating Mobile Applications Through Large Language Models

William Niemiec
william.niemiec@inf.ufrgs.br
Federal University of Rio Grande do Sul
Porto Alegre, Brazil

Anderson Rocha Tavares
artavares@inf.ufrgs.br
Federal University of Rio Grande do Sul
Porto Alegre, Brazil

Érika Fernandes Cota
erika@inf.ufrgs.br
Federal University of Rio Grande do Sul
Porto Alegre, Brazil

Abstract

The creation of applications using Large Language Models (LLMs) is a fast-growing area in software engineering. While the need for mobile applications increases, few studies have been conducted to investigate how to generate them through LLMs due to the complexity of handling the mobile ecosystem. In this work, we present and validate a platform-independent methodology for generating and evaluating complete mobile applications from high-level natural language descriptions. Our approach uses a multi-agent LLM framework to manage a three-stage process. First, a detailed mobile app description is generated from a document written in natural language. Next, our methodology converts the specification into executable source code. After that, we evaluate the implemented features and the source code quality. This methodology is built on important principles, including chain-of-thought and role-playing. Our findings show that the methodology produces compilable, stable, and crash-free applications with high-quality source code. However, the rates of feature implementation varied, with many functions being incomplete or missing altogether, which may point out the current limits of fully automated generation. This work gives a standardized process and a reproducible benchmark for evaluating LLM-generated applications. More broadly, this methodology enables the creation of functional prototypes from a high-level app description, thereby accelerating the development workflow.

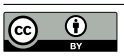
CCS Concepts

• **Computing methodologies** → **Natural language processing**;
• **Software and its engineering** → **Automatic programming**.

Keywords

mobile applications, software engineering process, large language models

ACM Reference Format:
William Niemiec, Anderson Rocha Tavares, and Érika Fernandes Cota. 2026. A Methodology for Generating Mobile Applications Through Large Language Models. In *IEEE/ACM 9th International Conference on Mobile Software Engineering and Systems (MOBILESoft '26), April 12, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3795077.3795116



MOBILESoft '26, Rio de Janeiro, Brazil

ACM ISBN 979-8-4007-2486-2/2026/04
https://doi.org/10.1145/3795077.3795116

1 Introduction

The global mobile application market has grown rapidly over the past decade, and it is expected to reach over 400 billion US dollars by 2026 [19]. These apps have become essential to daily life and are a key part of the digital economy. Consequently, this rising demand poses a big challenge for the software development industry [1]. The traditional process of developing a mobile application consumes a significant amount of resources, requiring specialized programming skills, substantial financial investment, and considerable time. This creates a bottleneck that slows innovation and limits access for individuals and small businesses.

In recent years, LLMs have emerged as a game-changing technology [6]. They demonstrate impressive capabilities in generating human-like text, code, and other complex media. This has allowed non-developer people to create advanced digital artifacts using natural language prompts. While researchers have studied the use of LLMs in many areas, there have been relatively few studies on fully automated generation of mobile applications. This research gap is due to the high complexity of handling mobile ecosystem issues, such as hardware elements, permissions handling, and operating system specificities. Also, these few studies present methods that often require complicated, multi-step processes that are still time-consuming and difficult for non-experts to navigate.

To address this gap, this paper proposes a methodology for creating and evaluating mobile applications made using LLMs. The main goal is to provide a standardized, reproducible methodology for generating and evaluating mobile application prototypes. While our approach simplifies the creation process, we investigate the extent to which current LLMs can go beyond scaffolding to produce functional logic. By benchmarking this capability, we aim to identify the boundaries that currently limit fully automated development.

To evaluate this capability, we introduce a benchmark based on reverse-engineering existing popular applications. Rather than relying on vague or subjective user prompts, we use descriptions from mobile app marketplaces as proxies for high-fidelity user requirements specifications. This allows us to measure the generated application against a known baseline, providing a clearer standard for evaluating the results.

The main contributions of this work are threefold:

- it proposes and validates a broad, platform-independent methodology for generating and benchmarking mobile applications using a Multi-Agent LLM Framework. This process, which goes from the initial user request to a detailed evaluation, aims to be easy for non-experts, greatly reducing the technical barrier to app creation.

- it introduces a new benchmark that specifically evaluates the quality of LLM-generated mobile applications. This two-part evaluation framework measures both functional quality (e.g., compile rate, feature implementation) and internal code quality (e.g., reliability, maintainability, complexity). It provides a standard and repeatable methodology for future research in this area.
- through a case study involving three diverse and popular applications (Bobbie Goods™, Duolingo™, and Threads™), we provided evidence that our methodology can produce compilable, executable, and stable application skeletons with a reasonable level of feature implementation and high-quality source code.

This paper is organized as follows. Section 2 reviews related work in mobile app development, evaluation, and LLM-based code generation. Section 3 outlines the proposed methodology for generating and evaluating applications. Section 4 describes the experimental setup created to test the methodology. Section 5 presents the results, while Section 6 discusses them, including their implications, limitations, and main findings. Finally, Section 7 wraps up with a summary of the main findings and possible directions for future research.

2 Related Work

This section reviews and discusses recent related work. First, we discuss related work regarding app description generation. Next, we present works related to code generation. Finally, we end this section showing works on evaluating the two previous topics: app descriptions and code generation.

2.1 App description generation through LLMs

The growing maturity of LLMs has created new opportunities for automated requirements engineering and feature elicitation. Wei et al. [25] conducted a study that looked at LLM-Inspiration, which uses GPT-4, and an AppStore [1] app as inspiration for feature refinement. This process involves breaking a broad feature down into a list of smaller sub-features. By manually reviewing 1,200 sub-features from 20 root features, the authors found that LLM-Inspiration is more effective, especially when examining new app ideas that had not been seen before. Additionally, the LLM approach showed very little overlap among the generated features. However, a significant challenge was the LLM's tendency to suggest features that were unrealistic or not possible because of technological limits or permission issues. This underscores the importance of having a human analyst in the elicitation process. On the other hand, while LLM-inspiration delivered relevant results for refining existing features, its effectiveness declined for new features because of the limitations of its app description database.

A different approach to using LLMs for generating app descriptions involves constructing features directly from non-code, non-text artifacts of applications. Peteliev et al. [17] looked into using LLMs to create app feature descriptions automatically from middle-level app artifact information. Their goal was to connect the claimed features with the app's real internal behavior. The researchers gathered various unstructured text strings from application bundle contents, such as entitlements, code-signing details, and file names from Resources and Helpers. They sent this data to the GPT-4o LLM through a customized prompt. The prompt instructed the model to create a readable list of four likely user tasks, requiring each description to begin with a verb and consist of 5 to 15 words, using only the provided artifact information. When comparing the descriptions created by the LLM to those written by humans, even basic prompting produced results that were "similar enough" to the human examples. They highlighted the potential of LLMs for generating automatic or assisted feature descriptions based on internal application data.

In our approach, we use a similar approach to Wei et al. [25]. We generate app descriptions from a natural language text written by a user, which can include a link to an app in AppStore[1], PlayStore[2], or other app stores. While Wei et al. [25] also start with a high-level feature description, their objective is hierarchical feature refinement (decomposing a feature into sub-features) using a single LLM model. In contrast, our approach processes the user input via a multi-agent LLM framework. This multi-agent architecture draws inspiration from related research [11, 15] regarding established prompt engineering techniques to improve LLM model outputs. The primary output of our method is a detailed app description, a comprehensive, high-level artifact, rather than just a list of structured sub-features or short user tasks.

2.2 Code generation through LLMs

The capabilities of LLMs in software development extend to code generation through images. de Souza Baulé et al. [5] performed a systematic study to identify approaches that generate code from images. The technical pipeline for this process typically employs a two-stage deep learning architecture. In the first stage, a Convolutional Neural Network (CNN) is used for object detection. Systems developed using this methodology have demonstrated promising results, achieving a precision of up to 80% for User Interface (UI) component detection and 60% accuracy for the final transformation into HTML/CSS code.

Several approaches use LLM to generate UI components. Yuan et al. [27] use LLM agents to generate HTML code from natural language. The authors conducted experiments comparing their method against baselines, and their findings confirm that preprocessing UI designs to extract structural features before LLM-based code generation leads to higher-quality outcomes. In a different way, some research has been developed for generating UI widgets, as DynaVis [23]. The paper presents a tool called DynaVis, which uses LLM to translate user commands into both the necessary visualization specification changes (i.e., changes needed to achieve a desired modification) and the HTML/JavaScript code for the dynamic widget. We also noted some researchers focused on studying evaluation and feedback of UI through natural language [2, 7, 8], and other studies addressing iterative refinement [16, 24]. These approaches utilize interactive loops, where users analyze, refine, and provide feedback based on the model's output, refining the solution until it meets expectations.

[1] https://www.apple.com/app-store

[2] https://play.google.com/store/apps

The literature demonstrates that while LLMs can generate high-quality, functionally correct code snippets and sophisticated methods exist for creating UIs from various inputs, there are few published studies on end-to-end generation of a complete, functional mobile application from a high-level textual description. This work seeks to fill this gap by using a multi-agent LLM to generate complete mobile applications from natural-language descriptions of how they should be.

2.3 Mobile app evaluation

The evaluation of a mobile application involves looking at its quality from different important perspectives. Features evaluation validates that the app's features meet the specified business and technical requirements. Kuznetsov et al. [14] introduce a technique to automatically detect mismatches between the advertised feature of an Android app and the functionality exposed through its UI. The core idea is to compare the topics present in an app's store description with the topics derived from the text within its UI elements. During their experiments, the authors found several discrepancies. For example, they discovered UI elements related to an in-app feature that was not mentioned in its store description at the time. The work demonstrated that statically analyzing UI text is a feasible method for identifying inconsistencies between an app's implementation and its public-facing claims.

Another evaluation one can perform in mobile apps is regarding code quality. High code quality is essential for long-term maintainability, reliability, and extensibility. Stefanović et al. [22] present a systematic literature review of static code analysis tools. The review analyzed 22 primary studies published over a ten-year period, highlighting the most commonly used analysis tools and which programming languages have the most support. They concluded that no single tool was found to detect all types of defects, and suggested that a combination of different tools would be required to achieve comprehensive defect detection. Expanding to other code quality criteria, Ivanković et al. [12] describe the implementation and adoption of a large-scale code coverage infrastructure at Google, which processes one billion lines of code across seven programming languages daily. Based on an analysis of five years of historical data and a survey of 3,000 developers, a significant number of developers reported finding coverage useful "Often" or "Very Often" when authoring (45%) and reviewing (40%) code changes. While their work primarily addresses dynamic test coverage, it establishes the necessity of automated, continuous assessment pipelines to maintain software reliability at scale.

3 Proposed Methodology

This section describes a methodology for creating and assessing mobile applications using LLMs. The main goal is to develop a process that is easy for non-experts by reducing the number of steps needed from the initial idea to the final app. By simplifying the technical details, the approach allows users to create functional applications from a natural language describing the features the app must have without specifying how to do that. The proposed workflow, shown in Figure 1, has three main stages: App Description Generation, Code Generation, and Evaluation. This setup systematically transforms a high-level user request into a functional mobile application that can be used and evaluated.

The following sections explain this methodology. Section 3.1 outlines the overall architecture and its main principles. Section 3.2 describes the methodology's phases. Finally, Section 3.3 presents the complete framework for evaluating the methodology's output.

3.1 Overall Architecture

Our approach is composed of two methodologies: app generation and evaluation strategy (Figure 1). The app generation methodology is managed by a Multi-Agent LLM Framework (called MALFRA hereafter), and uses a role specialization approach similar to Qian et al. [18]. While this framework orchestrates a sequential workflow between a "product manager" agent and a "software developer" one, the framework's core agentic capability lies in its capacity for iterative refinement with an automated feedback loop. This enables a crucial build-and-repair cycle where the agent framework does not simply execute a static script. If the initial code fails to compile or run, the framework captures errors, feeding them back into the code-generating agent's context. The agent then uses this information to produce a corrected version, automating a debugging process that would otherwise require manual intervention. This self-correction mechanism is a key differentiator from a simple, one-shot pipeline and is central to achieving a functional output.

The initial stage takes a user's natural language request and, using an LLM model, expands it into a detailed specification. The second stage uses this specification to generate the application's source code. The final stage evaluates the generated code against a set of functional and quality-based metrics. The entire process relies on carefully crafted prompts that leverage established prompt engineering techniques, such as role-playing [20] (e.g., "act as a senior software engineer") and chain-of-thought reasoning [26], to ensure clarity, context, and consistency for the LLM agents.

This architecture is defined by three key features:

- **Platform Agnosticism:** The methodology is designed to be independent of specific tools, ensuring its adaptability and long-term relevance. It allows for the integration of any multi-agent LLM framework (e.g., CrewAI[3], AutoGen[4]) and any underlying LLM model (e.g., GPT[5], Claude[6]). Similarly, the code generation stage can be configured to target various mobile development frameworks (e.g., React Native[7], Flutter[8]), providing the flexibility to meet diverse project requirements and adapt to the rapidly evolving landscape of LLM models and mobile development.
- **Hierarchical Decomposition:** The approach employs a top-down decomposition strategy, mirroring traditional software design principles. It begins with a high-level, abstract description of the desired application, such as a simple to-do list app, and progressively refines it into a detailed specification. This specification is then further broken down into

[3]https://github.com/crewAIInc/crewAI
[4]https://github.com/microsoft/autogen
[5]https://chatgpt.com
[6]https://claude.ai
[7]https://github.com/facebook/react-native
[8]https://github.com/flutter/flutter

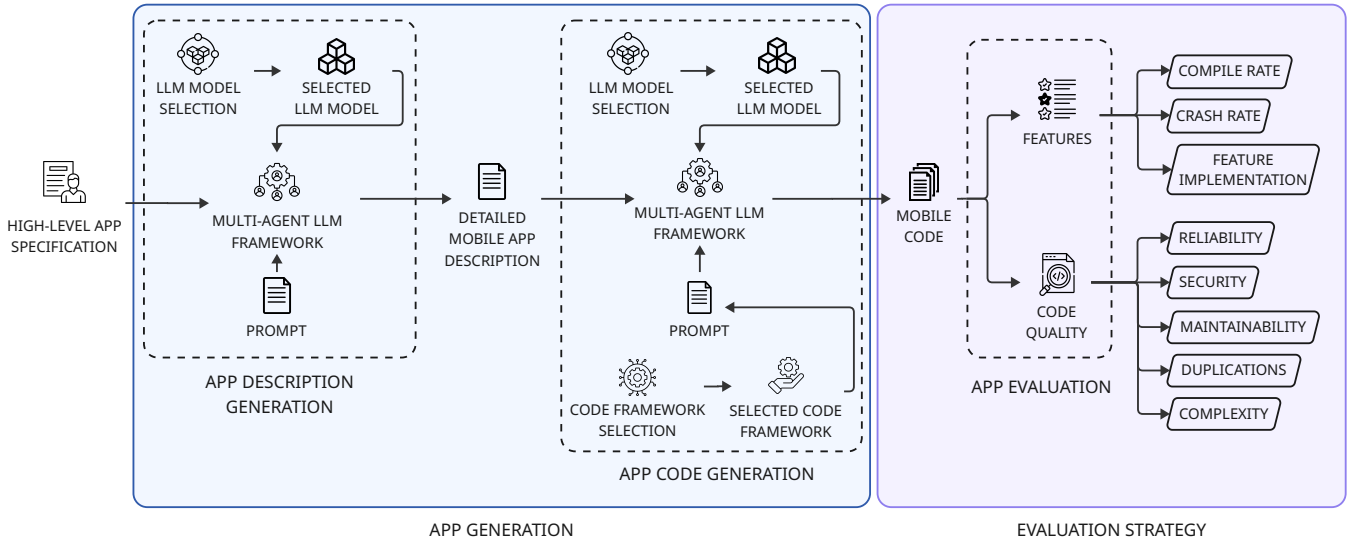


Figure 1: Proposed approach overview.

concrete, implementable source code for features like "add task", "view tasks", and "mark task as complete". This structured breakdown transforms a complex problem into smaller, manageable tasks, which is an ideal workflow for current LLMs.

- **Iterative Refinement with Feedback:** By leveraging a multi-agent LLM framework, the methodology incorporates an intrinsic, automated feedback loop. This enables a build-and-repair cycle where the LLM agents can iteratively review, debug, and refine the generated code. If the initial code fails to compile or run, the agent framework captures the compiler errors or runtime exceptions and feeds them back into the LLM's context. The LLM then uses this information to produce a corrected version, automating a significant part of the development and debugging process that would otherwise require manual intervention.

3.2 App Generation

The generation phase is a two-step process that turns an app idea into an executable mobile app. This phase involves carefully selecting two different but complementary components: a MALFRA and the underlying LLM models. The MALFRA serves as an orchestrator: it manages the overall workflow, defines agent roles, and automates the feedback loop (Figure 2). In contrast, the LLM model is a specialist or cognitive engine that performs specific tasks such as reasoning or coding. Choosing both is essential because the framework provides the structure for complex, multi-step operations, while selecting the right model allows for task-specific improvement. For example, one might choose a model with strong reasoning for a task and another with better coding skills for a different task.

In the first step, a carefully crafted prompt tells an LLM agent to act as a "product manager". The agent's job is to turn the user's broad request into a complete specification document. This document acts as the project's blueprint and may include a detailed feature list, use cases, UI design principles, the technology stack that should be used, and so on. To ensure deterministic code generation and precise evaluation, we explicitly instruct the agent to define the layout and components at a granular level. This includes listing specific UI elements (e.g., floating action button and login text field) and their intended screen locations. Consequently, the strategic role of UI specification is assigned to the App Description Generation agent, while the UI implementation is the sole responsibility of the App Code Generation agent. This separation of concerns ensures that the user's intent is fully captured and organized before any code is written.

After generating the initial description, the agents perform a programmatic audit to verify that all required schema sections are present and non-empty. If the output fails this structural check (e.g., missing the 'Layout' section), the agents perform a self-correction cycle, fixing the mismatch specifications until the strict formatting requirements are met. This prevents the propagation of incomplete specifications to the development phase.

In the second step, the mobile application code is generated from the detailed description. Also, another prompt is crafted, giving the previously-generated specification document to a "software developer" agent within the framework. It directs the agent to write the complete, production-ready source code for the application using a specified mobile framework. The agent then breaks down the tasks and uses the framework's iterative process to write, compile, and debug the code until a working version is produced. Note that the LLM model used to create a detailed app description may be different from the one used for generating code, as their goals are different: the first model creates a detailed mobile app specification, while the second one will generate mobile code, which allows the use of optimized models for each task.

3.3 Evaluation Strategy

To assess the effectiveness of the generation process, we propose an evaluation strategy that measures both the application's functional

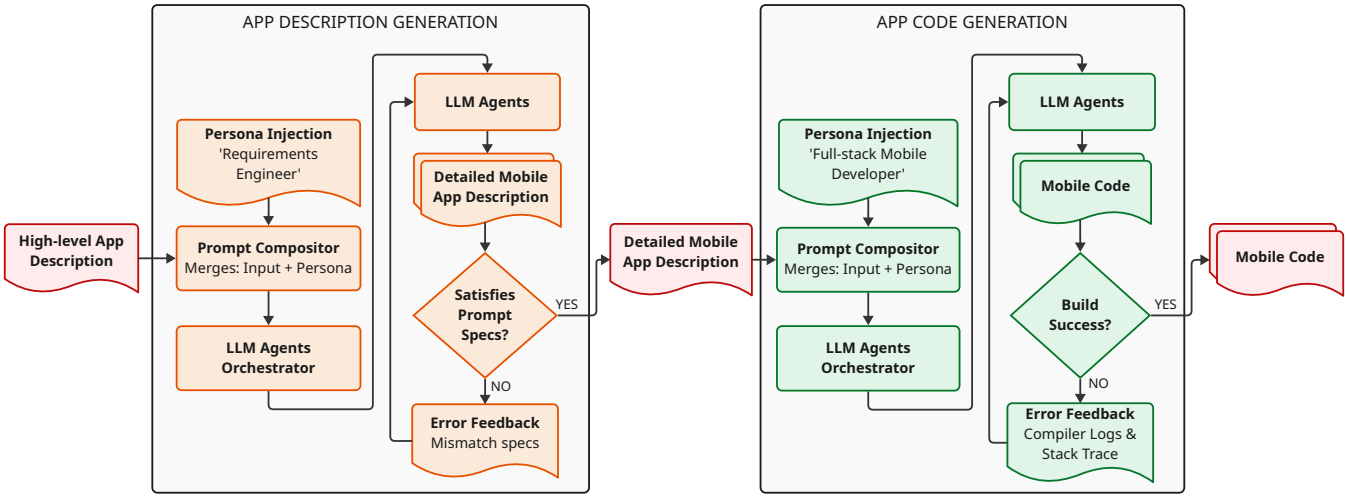


Figure 2: Multi-agent LLM Framework used in the "App Generation" phase.

quality and the technical quality of its source code. This structured evaluation constitutes a benchmark designed specifically for assessing LLM-generated mobile applications. We built it combining consolidated software evaluation techniques to analyze long-term code maintainability [21] and new ones to verify the app features that the LLM model correctly implemented.

3.3.1 Benchmark Workload and Usage Profile. We define the workload as the inputs and processes the system is subject to when measures are taken. The workload consists of two primary components to support independent replication:

- **Initial app description from a natural language** The set of diverse natural language user requests used to initiate the App Description Generation phase;
- **Interaction Profile (Execution Workload)** The predefined set of user interactions required for testing the mobile application during execution. This profile is crucial for determining the Crash Rate and the Feature Implementation Rate.

3.3.2 Essential Benchmark Qualities and Metrics. The benchmark is designed to quantify two key qualities, along with specific metrics and planned measurement methods: functional quality and source code quality. The detailed evaluated qualities and metrics are described in Table 1.

The proposed methodology uses LLM models in its construction. Due to the inherent stochastic nature of LLM outputs [4], evaluations using this methodology should include sufficient experiment repetitions and execution duration to ensure that the measured quality metrics are reliable and not artifacts of a single successful or failed generation attempt.

3.3.3 Testing Protocol for Functional Assessment. To ensure a comprehensive and repeatable assessment of features, a well-defined testing protocol is prescribed. This protocol translates the features outlined in the generated app description by the "project manager" agent into a series of exact user interactions. The process is as follows:

Table 1: Evaluation Metrics for Generated Mobile Applications. Note that the quality to be benchmarked is represented by feature qualities (i) and code qualities (ii).

Metric	Measurement Method
Compile Rate (i)	number of successes of the build process using the specified mobile framework tools, divided by the total number of times generated.
Crash Rate (i)	rate of fatal errors during execution of the predefined Interaction Profile.
Feature Implementation (i)	Granular assessment against requirements laid out in the LLM-generated specification document (generated by the "project manager" agent). Each feature in the document is tested via the Interaction Profile and categorized according to Table 2.
Reliability, Security, Maintainability, Duplications, Complexity (ii)	It may be measured via automated static analysis tools [22]. These tools quantify aspects like code smells, duplicate line ratio, and cyclomatic complexity.

(1) **Feature Extraction:** An evaluator systematically reviews the "Detailed Mobile App Description" document produced in the first stage of the methodology. Every distinct functional requirement (e.g., "user login", "add item to cart", "view user profile") is extracted and listed.
(2) **Test Case Derivation:** For each extracted feature, a specific sequence of user actions (a test case) is defined to validate its functionality. This includes specifying the exact inputs and expected outcomes.

For example, for an "add task" feature in a to-do list app, the test case would prescribe the following interactions:

- Tap the "Add New Task" button.

Table 2: Feature Implementation Categories

Category	Description
No Issues	The feature is fully functional and meets all requirements.
Minor Issues	The feature is functional but has minor defects that do not affect its core purpose (e.g., UI imperfections, placeholder data).
Major Issues	The feature is non-functional or contains severe bugs that inhibit its core purpose.
Not Implemented	The feature is entirely absent from the application.

- Enter "Buy groceries" into the text field.
- Tap the "Save" button.
- **Expected Outcome:** Verify that "Buy groceries" now appears in the main task list.

(3) **Protocol Compilation:** All individual test cases are compiled into a single, exhaustive testing script. This script serves as the complete *Interaction Profile* for the evaluator, guiding them through every feature test in a structured manner.

(4) **Execution and Classification:** A human expert evaluator executes the protocol on the generated application. As each feature's test case is performed, its outcome is recorded and classified according to the criteria in Table 2. This direct mapping ensures that the evaluation is systematic and directly measures functional completeness against the initial specification.

4 Experimental Setup

To validate the proposed methodology, we created and carried out an experimental setup following established software engineering benchmarking standards, which emphasize a structured and reproducible process for assessment [9]. These standards emphasize the need for a clear goal and context, a repeatable process, and the use of standard metrics. This ensures that the results are clear, trustworthy, and comparable within the research community.

Our experimental procedure consists of two main phases: Section 4.1 describes the specific configuration of our methodology, including the selection of application case studies, LLM models, the agent-based framework, and the mobile code framework; and Section 4.2 outlines the protocols and metrics used to assess the generated artifacts, covering both functional correctness and internal code quality.

4.1 App Generation

For our experiments, we chose a set of 3 applications. This number balances a variety of application types with the depth needed for thorough manual analysis. In order to have a consistent baseline, we generated apps based on existing ones. For that, we searched for applications and selected those that are compatible with Android devices, as it is the most used by people [10]. However, as we are using a code framework that can generate mobile applications for both Android and iOS, the code will be generated for both operating systems. The following formal criteria defined the selection process:

- **Inclusion Criteria:** The applications needed to be publicly available on the Google Play Store[1]. Also, the application must have a free plan and should be the most downloaded application in the category it belongs to when this study was performed (August 20, 2025). Note that this ranking was obtained directly from the Google Play Store[1], and it only reflects the most downloaded apps recently, and not from all time.
- **Exclusion Criteria:** We excluded applications for which success is not primarily defined by functional correctness, such as games.

We conducted our experiments by selecting Android apps from PlayStore[1]. Applying the inclusive criteria, we got 57 apps, each one belonging to a distinct category. Then, we excluded 23 apps for which success is not primarily defined by functional correctness. At the end, 34 applications remained, as shown in Figure 3. From them, we randomly selected 3 applications from different categories to ensure diversity in our experiments. The selected apps are described in Table 3.

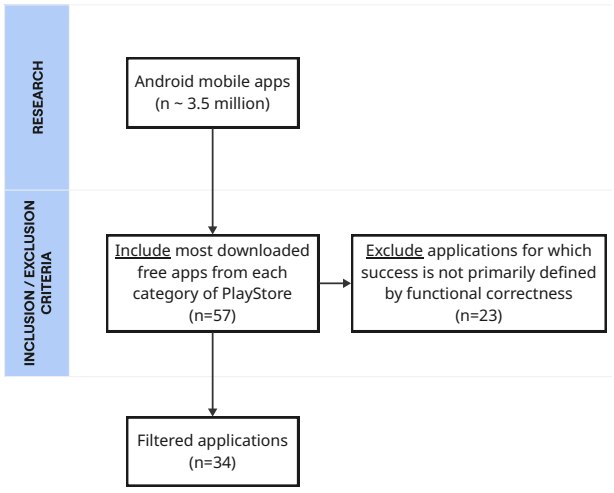


Figure 3: Mobile app selection.

To ensure the selection of the most capable models for each distinct task, we conducted a focused literature review of recent LLM evaluations and their performance on relevant, publicly available benchmarks. For the initial task of translating a high-level concept into a detailed technical specification, we selected Gemini 2.5 Pro[9]. This decision came from its high ranking on benchmarks like Aider[10], which evaluates an LLM's ability to understand and work within a specific codebase context. This metric is very relevant for our task because both require strong reasoning and the ability to produce clear, technically sound specifications. The model followed the structured prompt shown in Figure 4. Note that, for our experiments, we provided a link to a PlayStore app[1] instead of describing

[9]https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-5-pro
[10]https://doi.org/10.5281/zenodo.17425929

Table 3: Selected mobile applications to conduct our experiments

Name	Category	Description
Bobbie Goods™	Books	Allows users to color hundreds of hand-drawn, adorable scenes and characters from the Bobbie Goods™ universe on their smartphones or tablets.
Duolingo™	Education	A popular language learning app and website that uses a gamified format with quick, bite-sized lessons to teach over 40 languages.
Threads™	Social	A social media app designed for sharing text updates, photos, and videos, similar to X (formerly Twitter™).

an app. Although our methodology can be used to generate an app from a shallow description given by a user, we used a real app as a description because it would allow us to make a clear comparison of features from the generated mobile apps and the existing ones with minimum bias (e.g., one person could agree that the generated app is good and another not).

```
You are a skilled mobile app requirements engineer. Your task is to **generate a complete mobile app description** based on the provided app store's link.

**Steps:**
1. The app must meet all functional, design, and user-experience characteristics from the provided app store's link at **App store link**.
2. The final output should include the specified format at **Output Format:**.

**App store link:**
[LINK]

**Output Format:**

## App purpose
Description and objective of the app

## Detailed Feature List
List of detailed features to meet the requirements

## Layout and components
Full description of the app's screens, its layout, and the components of the screens

## User interaction flow
A complete description of all possible flows a user can have in the app
```

Figure 4: Prompt developed in this work to generate detailed app descriptions. Texts between brackets ("[" and "]") are placeholders to be replaced by the actual content in the prompt.

For the task of generating source code from the detailed description, we chose Claude 4.5 Sonnet[11]. We based this choice on having the best coding average performance on the LiveBench benchmark[12]. This benchmark is important because it tests models on previously unseen real-world coding problems. It serves as a reliable measure of a model's ability to produce high-quality, functional, and syntactically correct source code. The model was prompted with the details shown in Figure 5, where we included the complete output from the previous stage at the end of it.

```
You are an expert full-stack mobile application developer. Your task is to generate the complete source code, project structure, and setup documentation for a new mobile application, using the Flutter framework, based on the detailed specifications provided below:

[APP DESCRIPTION]
```

Figure 5: Prompt developed in this work to generate mobile app code from detailed app descriptions. Texts between brackets ("[" and "]") are placeholders to be replaced by the actual content in the prompt.

For the LLM agent-based framework selection, we conducted a market survey on MALFRA tools. The main requirement was that the tool must "generate apps through natural language". We then applied exclusion criteria to find tools that were accessible and flexible. This meant looking for tools that offered a free or freemium plan, were open-source, allowed integration of custom LLM models, and had a monthly subscription cost of $20 or less. The requirement for custom LLM integration was particularly critical, as our methodology is designed to be platform-agnostic and relies on selecting different, specialized models for the description and code generation tasks

Our initial search brought up 19 candidate tools, including online editors and IDE-integrated agents. Applying the inclusion requirement narrowed this down to 9 tools. After applying the exclusion criteria, we found just one tool that qualified: Kilo Code[13], as shown in Figure 6. As a Visual Studio Code extension, it met our needs for cost-effectiveness, open-source principles, and the flexibility to integrate our chosen LLMs. While our methodology is framework-agnostic (Figure 2), Kilo Code embodies this specific multi-agent architecture within an IDE environment.

At last, we had to choose a mobile code framework. This choice was based on the idea that an LLM's performance in code generation is linked to the amount of publicly available code it was likely trained on. We found that Flutter and React Native are the most common mobile code frameworks [13]. Next, we check which of them has more open source projects on GitHub. We found that the Flutter framework was the most popular, with approximately 960,000 repositories, significantly higher than React Native, which had roughly 561,000 open-source mobile framework repositories (accessed on October 22, 2025). Its ability to run on both iOS and Android from a single codebase also made it a strong choice for a method that could be used broadly.

[11] https://www.anthropic.com/news/claude-sonnet-4-5

[12] https://doi.org/10.5281/zenodo.17426044

[13] https://kilocode.ai

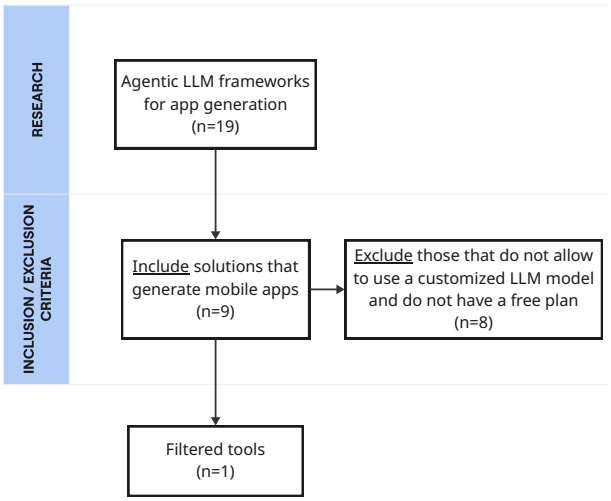


Figure 6: LLM agent-based framework selection.

4.2 Evaluation Strategy

The evaluation phase was carefully organized to examine two separate aspects of the produced artifacts. These aspects were the functional correctness of the application and the internal quality of its source code.

The functional capabilities of each generated application were evaluated via an expert review, conducted by a single evaluator (limitation further explained in Section 6.2). This evaluator executed a well-defined testing protocol, detailed in Section 3.3.3, which prescribed the exact user interactions required to test every feature outlined in the generated app description. This exhaustive protocol ensured a comprehensive assessment of functional completeness and correctness.

To perform a systematic and objective analysis of the source code's quality, we employed SonarQube[14], a widely adopted static analysis tool in both industry and academia. SonarQube was chosen for its comprehensive support for a wide range of code quality metrics and its capabilities for analyzing Flutter applications. The tool automatically evaluated the generated codebase against the metrics defined in our methodology. It is worth noting that we used the Community Edition of SonarQube, and while it is effective for code smells and bugs, it has limited sensitivity to deep security vulnerabilities. Thus, our security metrics should be interpreted as a baseline rather than a high-quality measure.

5 Results

This section presents the empirical results from the experimental validation of the proposed methodology. The three case study applications (Bobbie Goods™, Duolingo™, and Threads™) were generated and assessed according to the evaluation framework defined in Section 3.3. The findings are presented in two parts: functional quality assessment (Section 5.1) and source code quality assessment (Section 5.2), providing a comprehensive view of the methodology's output.

[14]https://www.sonarsource.com/products/sonarqube

5.1 Functional Quality Assessment

The functional quality of each generated application was evaluated against our benchmark's metrics: Compile Rate, Crash Rate, and Feature Implementation. Initial build errors were automatically resolved through the iterative refinement loop within the agentic framework. Also, we repeated the app generation flow three times for each application to ensure the stability of our LLM model, and the presented results are the average of these iterations. Two of the generated applications (Bobbie Goods™ and Threads™) compiled successfully in all three iterations, while one of them (Duolingo™) only compiled in one iteration, requiring an average of 3 iterations to produce a stable build.

After generating each app, the next step is to evaluate it. We conducted a granular assessment to determine the degree to which the generated applications met the functional requirements specified in the "Detailed Mobile App Description" (Section 3.2). Each feature was systematically tested and classified using the criteria outlined in Table 2. At the end, we had 21 features related to Bobbie Goods™, 18 to Duolingo™, and 12 to Threads™. The results are shown in Table 4 and Figure 7.

Table 4: Functional quality results

Metric	Bobbie Goods™	Duolingo™	Threads™
Compile Rate	100%	33.33%	100%
Crash Rate	0.0%	0.0%	0.0%
No Issues	3	7	5
Minor Issues	0	0	2
Major Issues	3	3	1
Not Implemented	5	8	4
Total Features	11	18	12

Overall, the results indicate that the proposed methodology produces interactive, high-fidelity UI prototypes with limited human intervention. While the MALFRA consistently generated compilable and executable application skeletons, we acknowledge that being compilable does not equate to full functionality. As shown in Table 4, the Crash Rate remained at 0%; however, this stability is partially attributed to the fact that many complex, data-driven features remained unimplemented. In these cases, the absence of backend logic or state management naturally reduces the probability of runtime fatal errors. Consequently, while the generated artifacts are stable, they currently represent high-fidelity skeletons rather than production-ready applications.

Regarding feature completeness, Threads™ performed best, with over 58% of its functionalities either fully implemented or implemented with minor issues. Bobbie Goods™ and Duolingo™ achieved 27.27% and 38.89%, respectively. The proportion of unimplemented or severely incomplete features highlights the potential limitations of the MALFRA in generating full mobile applications directly (i.e., using only one prompt).

Our methodology demonstrated high efficacy in generating self-contained utility features and basic UI elements. Features that were successfully implemented shared a common characteristic: they

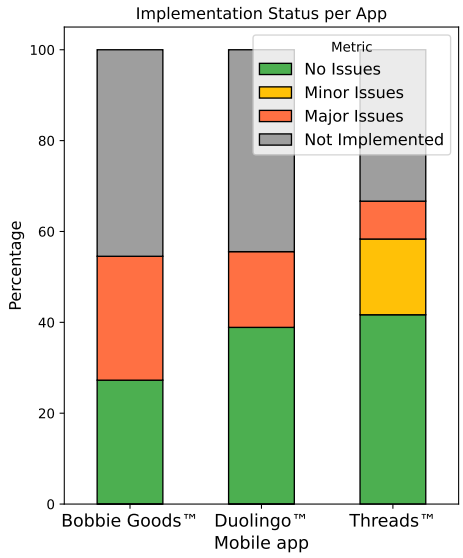


Figure 7: Stacked bar chart showing feature qualities over the tested applications.

were largely self-contained, stateless, or boilerplate UI components. This category includes common utility functions (like view manipulation), static content displays, and simple data trackers. This suggests the methodology is proficient at scaffolding basic application structures and implementing standard, reusable components that are decoupled from core logic.

Several features were classified as having "Minor" or "Major Issues". We identified a pattern in our experiments: the common trait in "Minor Issues" features was a failure in the finer details of simple state management or data binding. Our approach correctly generated the UI for these interactions (e.g., buttons or currency displays) but faltered in the backend logic required to persist state changes across the application. On the other hand, features classified as "Major Issues" consistently included data-driven features requiring aggregation and comparison (like leaderboards), personalized or algorithmic content feeds, and any feature requiring the management of a complex, interactive application state. The core challenge MALFRA faced was handling complex backend logic, which it addressed by stubbing backend calls.

The most severe limitations of our methodology are exposed by the "Not Implemented" category. This category consistently included the core-domain frameworks (such as the educational lesson structure or the social graph integration), critical subsystems like monetization, and the essential architectural components that define an application's unique purpose.

5.2 Code Quality Assessment

Beyond functional correctness, the quality of the source code was measured via automated static analysis using the SonarQube tool. This analysis provided objective metrics for key software engineering attributes, including reliability, security, maintainability, and complexity, as defined in our evaluation methodology. The results of this analysis are presented in Table 5.

Table 5: Code quality results

Metric	Bobbie Goods™	Duolingo™	Threads™
Reliability (bugs)	0	0	0
Security (vulnerabilities)	0	0	0
Maintainability (code smells per file)	2.16	1.68	1.74
Duplicated Lines	2.4%	0.0%	12.2%
Cyclomatic Complexity (per function)	2.23	1.30	1.60
Files	19	19	23
Functions	301	116	376

In summary, the code quality analysis revealed promising results. All generated apps exhibited zero detected bugs or security vulnerabilities, showing that the MALFRA framework followed safe and syntactically correct coding patterns. Maintainability scores varied moderately, with SonarQube detecting from approximately 2.16 to 1.68 code smells per file. Duplication rates remained low (<13%), and complexity metrics were within acceptable bounds (up to 10 [3]).

6 Discussion

This section provides context for our findings by discussing them, recognizing the limitations of our study, and outlining the practical implications for both researchers and practitioners.

6.1 Key findings

The results showed how our proposed methodology can automatically generate functional mobile apps from high-level natural language descriptions. The successful compilation and execution of the case study applications, along with their stable, crash-free performance, confirm the basic viability of this approach. In addition, the code quality analysis reveals that the generated artifacts follow good software engineering practices, as they do not have any critical bugs or security issues and keep a manageable level of complexity. However, the different levels of feature completeness indicate that while this methodology is an important step forward, we are not yet at the point of completely autonomous, production-ready app generation.

6.2 Limitations and Threats to validity

A qualitative analysis of the "Major Issues" and "Not Implemented" categories reveals a distinct boundary in current LLM capabilities: the backend hallucination gap and the state management complexity.

For apps like Threads™ and Duolingo™, core functionality relies on proprietary backend algorithms. The LLM agents correctly identified these requirements, but could not implement them because they require server-side infrastructure beyond the scope of

the mobile-only code generation. The agents often generated "stub" methods or empty UI states where this data should appear.

Also, while the agents successfully implemented stateless UI components (screens, navigation), they struggled with complex state management across multiple screens. This suggests that the context window or reasoning capability is sufficient for view generation but insufficient to maintain a coherent application-state architecture without explicit architectural prompts.

At last, we identified three threats to the validity of this study, which are discussed next.

- **Internal validity:** Our functional evaluation was performed by using a single expert evaluator. This could introduce subjective bias in assessing feature correctness and classifying implementation issues. Future work will expand this by adopting a large number of evaluators. Also, the stochastic nature of LLM models may limit the reproducibility of the experiments. We mitigated this risk by repeating the generation process three times for each app to capture the variation in LLM model outputs. Ultimately, the detailed app description prompt can significantly impact the results of the generated mobile apps. This risk was mitigated through the adoption of prompt engineering techniques, including role-playing [20] and chain-of-thought reasoning [26].
- **External validity:** The main threat to generalization is the small sample size. Our findings are based on generating only three categories of apps, which, although diverse, may not represent all mobile apps. Future work should test the methodology on a broader range of applications. Additionally, our experiments were limited to one framework, and the results may not directly apply to native iOS/Android development or other cross-platform frameworks. We mitigate this risk by choosing the mobile app framework that has the most open-source availability on the internet, as the LLM models were more exposed to these codes during their training. At last, our results are snapshot-dependent on the specific LLM versions (e.g., Gemini 2.5 Pro and Claude 4.5 Sonnet). As these models evolve monthly, our contribution is the methodology framework, not the specific benchmarks of these model versions.
- **Construct validity:** A limitation of our evaluation strategy is its reliance on the generated "Detailed Mobile App Description" as the ground truth. Our current metrics measure the fidelity of the implementation relative to the generated specification, rather than the fidelity of the specification relative to the original high-level app specification. Consequently, if the App Description Generation agent omits a critical feature from the description, the subsequent evaluation will not consider this as a failure, provided the code perfectly matches the incomplete description. Future work should investigate how to mitigate this risk.

6.3 Practical Implications

The proposed methodology allows individuals without specialized programming skills, like entrepreneurs, designers, and small business owners, to turn their ideas into functional prototypes. This can significantly reduce the initial time and financial investment needed to validate a concept.

For professional software engineers, this approach can be a strong productivity booster. It can automate the generation of boilerplate code and basic features, enabling developers to concentrate on more complex, creative, and high-value tasks like refining the user interface, optimizing performance, and implementing sophisticated business logic.

When you compare the proposed approach to current no-code and low-code platforms, there are clear advantages and trade-offs. Traditional no-code tools use pre-defined UI components and rule-based workflows. These tools give users a lot of control over design, but limit flexibility for expanding functionality beyond what is predefined. On the other hand, our approach creates both the application specification and source code (Figure 1). This allows the final output to be fully modifiable, portable, and usable in any software development pipeline. Developers can extend the generated code using standard programming techniques without being limited by platform-specific abstractions.

7 Conclusion

The rapid growth of the mobile application market has created a significant demand for more efficient and accessible development processes. This paper addressed this challenge by proposing and validating a new methodology for generating and evaluating mobile applications using LLM models. Our three-stage approach (Figure 1) uses MALFRA to turn a high-level user request into a functional application with minimal human input.

Our experimental validation, conducted on three diverse, real-world applications, shows the feasibility and potential of this methodology. It consistently produced applications that were compilable, executable, and stable, with a zero crash rate across all tests. Static analysis of the generated source code showed high quality, with no reliability bugs or security vulnerabilities found, and maintainability metrics within acceptable limits. However, the results also highlighted limitations, with varying feature implementation rates and a notable percentage of functionalities remaining unimplemented or having major issues.

Future work will test the methodology across a broader, more complex range of applications. This expansion should include a larger set of evaluators to mitigate the risk of subjective bias from a single reviewer. We also plan to test various MALFRA configurations. Ultimately, we plan to conduct a user study in which participants interact with the generated prototypes to assess their readiness and perceived utility relative to the original specification. This will help determine whether end-users perceive the unimplemented features as critical blockers.

Acknowledgments

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001. This work is also partially supported by Propesq-UFRGS.

Data availability

The data that support the findings of this study are openly available in Zenodo at https://doi.org/10.5281/zenodo.17353920.

References

[1] Arshad Ahmad, Kan Li, Chong Feng, Syed Mohammad Asim, Abdallah Yousif, and Shi Ge. 2018. An empirical study of investigating mobile applications development challenges. *IEEE Access* 6 (2018), 17711–17728.

[2] Wajdi Aljedaani, Marcelo Medeiros Eler, and PD Parthasarathy. 2025. Enhancing accessibility in software engineering projects with large language models (llms). In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*. Association for Computing Machinery, New York, NY, USA, 25–31.

[3] Paul Ammann and Jeff Offutt. 2017. *Introduction to software testing*. Cambridge University Press, USA.

[4] Xiang Chen, Chaoyang Gao, Chunyang Chen, Guangbei Zhang, and Yong Liu. 2025. An empirical study on challenges for llm application developers. *ACM Transactions on Software Engineering and Methodology* 34, 7, Article 205 (2025), 37 pages.

[5] Daniel de Souza Baulé, Christiane Gresse von Wangenheim, Aldo von Wangenheim, and Jean CR Hauck. 2020. Recent Progress in Automated Code Generation from GUI Images Using Machine Learning Techniques. *J. Univers. Comput. Sci.* 26, 9 (2020), 1095–1127.

[6] Xiaofei Dong, Xueqiang Zhang, Weixin Bu, Dan Zhang, and Feng Cao. 2024. A survey of llm-based agents: Theories, technologies, applications and suggestions. In *2024 3rd International Conference on Artificial Intelligence, Internet of Things and Cloud Computing Technology (AIoTC)*. IEEE, IEEE, New York, NY, USA, 407–413.

[7] Peitong Duan, Jeremy Warner, Yang Li, and Bjoern Hartmann. 2024. Generating automatic feedback on ui mockups with large language models. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 1–20.

[8] Leiser et al. 2024. Hill: A hallucination identifier for large language models. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 1–13.

[9] Ralph et al. 2020. Empirical Standards for Software Engineering Research. arXiv:2010.03525 [cs.SE] https://arxiv.org/abs/2010.03525

[10] Semra Gündüç and Recep Eryiğit. 2018. Role of new ideas in the mobile phone market share. *International Journal of Modeling, Simulation, and Scientific Computing* 9, 02 (2018), 1850018.

[11] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30.

[12] Marko Ivanković, Goran Petrović, René Just, and Gordon Fraser. 2019. Code coverage at Google. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 955–963.

[13] Gregor Jošt and Viktor Taneski. 2025. State-of-the-Art Cross-Platform Mobile Application Development Frameworks: A Comparative Study of Market and Developer Trends. *Informatics* 12, 2 (2025), 1–30. doi:10.3390/informatics12020045

[14] Konstantin Kuznetsov, Vitalii Avdiienko, Alessandra Gorla, and Andreas Zeller. 2016. Checking app user interfaces against app descriptions. In *Proceedings of the International Workshop on App Market Analytics*. Association for Computing Machinery, New York, NY, USA, 1–7.

[15] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. 2024. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth* 1, 1 (2024), 9.

[16] Damien Masson, Sylvain Malacria, Géry Casiez, and Daniel Vogel. 2024. Directgpt: A direct manipulation interface to interact with large language models. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 1–16.

[17] Yevhenii Peteliev, Ivan Synytsia, and Nataliia Stulova. 2025. Towards Generating App Feature Descriptions Automatically with LLMs: the Setapp Case Study. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE, IEEE Computer Society, Los Alamitos, CA, USA, 252–256.

[18] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 15174–15186. doi:10.18653/v1/2024.acl-long.810

[19] Grand View Research. 2023. Mobile Application Market Size, Share & Trends Analysis Report By Store (Google Store, Apple Store, Others), By Application (Gaming, Music & Entertainment, Health & Fitness, Social Networking), And Region, Segment Forecasts, 2024 - 2030. https://www.grandviewresearch.com/industry-analysis/mobile-application-market. Accessed: 2025-09-24.

[20] Murray Shanahan, Kyle McDonell, and Laria Reynolds. 2023. Role play with large language models. *Nature* 623, 7987 (2023), 493–498.

[21] I. Sommerville. 2011. *Software Engineering*. Pearson, London, UK. https://books.google.com.br/books?id=l0egcQAACAAJ

[22] Darko Stefanović, Danilo Nikolić, Dusanka Dakic, Ivana Spasojević, and Sonja Ristic. 2020. *Static Code Analysis Tools: A Systematic Literature Review*. DAAAM International, Vienna, Austria, 0565–0573. doi:10.2507/31st.daaam.proceedings.078

[23] Priyan Vaithilingam, Elena L Glassman, Jeevana Priya Inala, and Chenglong Wang. 2024. Dynavis: Dynamically synthesized ui widgets for visualization editing. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 1–17.

[24] Zijie J Wang, Chinmay Kulkarni, Lauren Wilcox, Michael Terry, and Michael Madaio. 2024. Farsight: Fostering responsible ai awareness during ai application prototyping. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 1–40.

[25] Jialiang Wei, Anne-Lise Courbis, Thomas Lambolais, Binbin Xu, Pierre Louis Bernard, Gerard Dray, and Walid Maalej. 2024. Getting inspiration for feature elicitation: app store-vs. LLM-based approach. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, New York, NY, USA, 857–869.

[26] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 1800, 14 pages.

[27] Dawei Yuan, Guocang Yang, and Tao Zhang. 2025. UI2HTML: utilizing LLM agents with chain of thought to convert UI into HTML code. *Automated Software Engineering* 32, 2 (2025), 1–24.